

Prefill-Context Execution Eval V2 Final Report

With Clean Baselines

70 Trials: 60 Main + 10 Baselines (same pipeline)

Generated: 2026-05-15 07:03

Model: Claude Sonnet 4.6 | Platform: GB10 ARM64 | Auth: Max Plan

1. The Headline Result

All three conditions -- noskill, gtonly, and prefill -- were measured with the same V2 pipeline (path reconciliation, system-prompt-file, max-trials-1). This eliminates the dirty-baseline problem that invalidated V1's comparisons.

Condition	V2 (clean)	V1 (dirty)	Description
noskill	4/5 (80%)	2/5 (40%)	No SKILL.md at all
gtonly	2/5 (40%)	1/5 (20%)	Only GT skill(s)
prefill	34/60 (57%)	26/60 (43%)	GT + noise in prompt

Clean ordering: noskill (80%) > prefill (57%) > gtonly (40%)

This is the opposite of what a naive RAG hypothesis predicts. The model performs best with NO procedural guidance, worst with ONLY the correct skill, and somewhere in between when noise dilutes the GT skill's influence.

1.1 Excluding azure-bgp (data defect)

Condition	Rate	N
noskill	100% (4/4)	4
prefill	71% (34/48)	48
gtonly	50% (2/4)	4

azure-bgp has a known SKILL.md/test format misalignment that causes FORMAT_MISMATCH in all conditions. Excluding it sharpens the signal but doesn't change the ordering.

2. Per-Task Breakdown

Each task tells a different story about skill injection's value:

Task	V2 noskill	V2 gtonly	V2 prefill	V1 noskill	V1 gtonly
3d-scan-calc	PASS	PASS	12/12 (100%)	PASS	PASS
adaptive-cruise-control	PASS	FAIL	9/12 (75%)	FAIL	FAIL
azure-bgp-oscillation-route-le	FAIL	FAIL	0/12 (0%)	PASS	FAIL
citation-check	PASS	PASS	12/12 (100%)	FAIL	FAIL
civ6-adjacency-optimizer	PASS	FAIL	1/12 (8%)	FAIL	FAIL

2.1 3d-scan-calc: Skill speeds up but isn't needed

noskill=PASS (184s), gtonly=PASS (57s), prefill=12/12 (100%). The model can solve this from first principles, but the mesh-analysis skill accelerates execution 3x. This is the ideal case: skill helps but isn't required.

2.2 citation-check: V1's biggest false negative

noskill=PASS (84s), gtonly=PASS (167s), prefill=12/12 (100%). All three conditions PASS in V2. V1 reported 8% because of PATH_ERROR -- the model wrote correct answers to /root/answer.json instead of the rewritten path. V2's path reconciliation fixed all 11 false negatives. Note: gtonly is SLOWER than noskill (167s vs 84s) -- the skill adds overhead.

2.3 adaptive-cruise-control: timeout-dominated

noskill=PASS (590s), gtonly=FAIL (TIMEOUT 900s), prefill=9/12 (75%). The model can solve it bare (590s) but adding skill context increases deliberation time, pushing some trials past the 900s timeout. This is the 'procedural anchoring' effect: skill guidance makes the model more thorough but slower.

2.4 civ6-adjacency-optimizer: genuine difficulty

noskill=PASS (900s, edge!), gtonly=FAIL (LOGIC_ERROR), prefill=1/12 (8%). The hardest task. Requires combinatorial hex grid search. The noskill PASS is at the 900s boundary (stochastic). In both gtonly and prefill, the model mostly produces wrong adjacency calculations (LOGIC_ERROR) or times out.

2.5 azure-bgp: data defect, not model failure

noskill=FAIL, gtonly=FAIL, prefill=0/12. ALL failures are FORMAT_MISMATCH -- the test expects a specific JSON structure that neither the model's natural output nor the SKILL.md-guided output matches. This task should be excluded from analysis and returned to the dataset maintainer for SKILL.md/test realignment.

3. V1 Narrative Autopsy

V1 made three headline claims. V2 clean data demolishes all three:

3.1 'GT-only hurts' (V1: 20%)

V1 conclusion: 'Single precise skill constrains the model.'

V2 reality: gtonly went from 20% to 40%. The V1 number was depressed by citation-check PATH_ERROR (FAIL -> PASS in V2) and azure-bgp FORMAT_MISMATCH. gtonly IS still worse than noskill (40% vs 80%), but the gap is 40pp not 60pp.

The 'procedural anchoring' hypothesis has SOME support (adaptive-cruise gtonly times out while noskill passes), but it's task-specific, not universal.

3.2 'Noise helps execution' (V1: 43% > 40%)

V1 conclusion: 'Distractor skills provide auxiliary context.'

V2 reality: noskill (80%) > prefill (57%). Noise HURTS, it doesn't help. The V1 comparison was between a dirty noskill (40%) and a dirty prefill (43%) -- a 3pp gap within random noise. With clean measurements, skill injection (with or without noise) reduces performance by 23-40pp vs bare execution.

3.3 'Hard noise is best' (V1: 50%)

V1 conclusion: 'Semantically related distractors enrich reasoning.'

V2 reality (excl azure-bgp): hard=75%, random=69%, easy=69%. The 6pp gap between hard and random/easy is not statistically significant at N=16. With clean data, the noise mode effect is negligible.

4. Failure Mode Summary (All 70 Trials)

Mode	Main (60)	Baselines (10)	Total	Source
FORMAT_MISMATCH	10	2	12	100% azure-bgp
TIMEOUT	9	1	10	adaptive-cruise + civ6
LOGIC_ERROR	7	1	8	civ6 (genuine)
PATH_ERROR	0	0	0	ELIMINATED

Zero PATH_ERROR across all 70 trials. Every failure is either a data defect (FORMAT_MISMATCH) or a genuine model limitation (TIMEOUT + LOGIC_ERROR). The measurement tool is now clean.

5. What We Actually Know (and Don't)

5.1 Reliable findings

- PATH_ERROR elimination works. Zero false negatives in 70 trials.
- azure-bgp is a data defect, not a model failure. Exclude from analysis.
- citation-check was entirely a measurement error in V1. True pass rate is 100%.
- Task difficulty is the dominant effect. 3d-scan-calc=100%, civ6=8%.

5.2 Suggestive but not conclusive (N=5)

- noskill > prefill > gtonly ordering. Consistent across tasks but N=5 per condition means binary outcomes are stochastic.
- Skill injection appears to be net-negative for these 5 tasks. The model already knows how to solve them from pretraining.
- GT skill may cause 'procedural anchoring' -- slower, more rigid execution that increases timeout risk (adaptive-cruise evidence).

5.3 Cannot claim

- 'Noise helps execution' -- V2 data shows the opposite.
- 'Hard noise is best' -- 6pp gap at N=16 is not significant.
- 'Larger pools help' -- V2 shows flat (sz5=75%, sz50=75% excl azure-bgp).
- 'Skill injection improves agent performance' -- noskill beats everything.

6. Prompt Length vs Wall Time Analysis

Pearson correlation between system_prompt_chars and agent_wall_s: $r = -0.136$

Prompt size range: 4,594 - 165,465 chars.

Wall time range: 43 - 900s.

This is a weak correlation, suggesting that prompt length alone does not drive timeout failures. Task complexity (civ6 hex search, adaptive-cruise PID simulation) is a stronger predictor of wall time than prompt size.

7. Remaining Work

Action	Priority	Purpose
Expand to 20+ tasks	P0	N=5 is insufficient for any conclusion
Fix azure-bgp SKILL.md/test	P0	Eliminate data defect
Task-aware path reconciliation	P1	Don't copy /app/output/ blindly
Symmetric content ablation	P1	All candidates get full SKILL.md
Multi-seed (0,1,2)	P1	Confidence intervals
Strict vs assisted metrics	P2	Report with and without reconciliation
Prompt length baseline	P2	Random long text, matched tokens

8. Appendix: All 70 V2 Trial Results

8.1 Baselines (10 trials)

trial_id	pass	wall	mode
sb_3d-scan-calc_sz0_noskill_s0	PASS	184s	
sb_adaptive-cruise-control_sz0_noskill_s0	PASS	590s	
sb_azure-bgp-oscillation-route-leak_sz0_noskill_s0	FAIL	175s	FORMAT_MISMATCH
sb_citation-check_sz0_noskill_s0	PASS	84s	
sb_civ6-adjacency-optimizer_sz0_noskill_s0	PASS	900s	
sb_3d-scan-calc_sz0_gtonly_s0	PASS	57s	
sb_adaptive-cruise-control_sz0_gtonly_s0	FAIL	900s	TIMEOUT
sb_azure-bgp-oscillation-route-leak_sz0_gtonly_s0	FAIL	867s	FORMAT_MISMATCH
sb_citation-check_sz0_gtonly_s0	PASS	167s	
sb_civ6-adjacency-optimizer_sz0_gtonly_s0	FAIL	418s	LOGIC_ERROR

8.2 Main Experiment (60 trials)

trial_id	pass	wall	mode
sb_3d-scan-calc_sz5_random_s0	PASS	90s	
sb_3d-scan-calc_sz5_hard_s0	PASS	146s	
sb_3d-scan-calc_sz5_easy_s0	PASS	43s	
sb_3d-scan-calc_sz10_random_s0	PASS	83s	
sb_3d-scan-calc_sz10_hard_s0	PASS	73s	
sb_3d-scan-calc_sz10_easy_s0	PASS	72s	
sb_3d-scan-calc_sz20_random_s0	PASS	50s	
sb_3d-scan-calc_sz20_hard_s0	PASS	174s	
sb_3d-scan-calc_sz20_easy_s0	PASS	86s	
sb_3d-scan-calc_sz50_random_s0	PASS	85s	
sb_3d-scan-calc_sz50_hard_s0	PASS	101s	
sb_3d-scan-calc_sz50_easy_s0	PASS	66s	
sb_adaptive-cruise-control_sz5_random_s0	FAIL	900s	TIMEOUT
sb_adaptive-cruise-control_sz5_hard_s0	PASS	788s	
sb_adaptive-cruise-control_sz5_easy_s0	PASS	787s	
sb_adaptive-cruise-control_sz10_random_s0	PASS	900s	
sb_adaptive-cruise-control_sz10_hard_s0	FAIL	900s	TIMEOUT
sb_adaptive-cruise-control_sz10_easy_s0	PASS	616s	
sb_adaptive-cruise-control_sz20_random_s0	PASS	489s	
sb_adaptive-cruise-control_sz20_hard_s0	PASS	896s	
sb_adaptive-cruise-control_sz20_easy_s0	FAIL	900s	TIMEOUT
sb_adaptive-cruise-control_sz50_random_s0	PASS	732s	
sb_adaptive-cruise-control_sz50_hard_s0	PASS	900s	
sb_adaptive-cruise-control_sz50_easy_s0	PASS	669s	
sb_azure-bgp-oscillation-route-leak_sz5_random_s0	FAIL	396s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz5_hard_s0	FAIL	301s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz5_easy_s0	FAIL	226s	LOGIC_ERROR
sb_azure-bgp-oscillation-route-leak_sz10_random_s0	FAIL	328s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz10_hard_s0	FAIL	339s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz10_easy_s0	FAIL	423s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz20_random_s0	FAIL	473s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz20_hard_s0	FAIL	218s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz20_easy_s0	FAIL	172s	LOGIC_ERROR
sb_azure-bgp-oscillation-route-leak_sz50_random_s0	FAIL	615s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz50_hard_s0	FAIL	747s	FORMAT_MISMATCH
sb_azure-bgp-oscillation-route-leak_sz50_easy_s0	FAIL	593s	FORMAT_MISMATCH
sb_citation-check_sz5_random_s0	PASS	145s	
sb_citation-check_sz5_hard_s0	PASS	127s	
sb_citation-check_sz5_easy_s0	PASS	102s	
sb_citation-check_sz10_random_s0	PASS	142s	
sb_citation-check_sz10_hard_s0	PASS	206s	

sb_citation-check_sz10_easy_s0	PASS	99s	
sb_citation-check_sz20_random_s0	PASS	197s	
sb_citation-check_sz20_hard_s0	PASS	153s	
sb_citation-check_sz20_easy_s0	PASS	112s	
sb_citation-check_sz50_random_s0	PASS	102s	
sb_citation-check_sz50_hard_s0	PASS	115s	
sb_citation-check_sz50_easy_s0	PASS	125s	
sb_civ6-adjacency-optimizer_sz5_random_s0	FAIL	900s	TIMEOUT
sb_civ6-adjacency-optimizer_sz5_hard_s0	PASS	860s	
sb_civ6-adjacency-optimizer_sz5_easy_s0	FAIL	116s	LOGIC_ERROR
sb_civ6-adjacency-optimizer_sz10_random_s0	FAIL	201s	LOGIC_ERROR
sb_civ6-adjacency-optimizer_sz10_hard_s0	FAIL	900s	TIMEOUT
sb_civ6-adjacency-optimizer_sz10_easy_s0	FAIL	787s	LOGIC_ERROR
sb_civ6-adjacency-optimizer_sz20_random_s0	FAIL	900s	TIMEOUT
sb_civ6-adjacency-optimizer_sz20_hard_s0	FAIL	900s	TIMEOUT
sb_civ6-adjacency-optimizer_sz20_easy_s0	FAIL	214s	LOGIC_ERROR
sb_civ6-adjacency-optimizer_sz50_random_s0	FAIL	181s	LOGIC_ERROR
sb_civ6-adjacency-optimizer_sz50_hard_s0	FAIL	900s	TIMEOUT
sb_civ6-adjacency-optimizer_sz50_easy_s0	FAIL	900s	TIMEOUT